Multi-type Populations

EdgeBasedModels.jl Vignette 05

Simon Frost

2026-05-14

Table of contents

Introduction

Real populations are rarely homogeneous. Age groups, risk groups, spatial patches, and other forms of heterogeneity lead to structured contact patterns where individuals of different types mix at different rates. Multi-type edge-based compartmental models extend the EBCM framework to track the epidemic on each type of edge separately.

For K population types and M disease stages, the package automatically generates $K^2(1+M)+KM$ ODEs — the K^2 comes from tracking θ_{jl} (probability that an edge from type l to type j has not transmitted) for every ordered pair of types, K^2M from the ϕ variables for each disease stage and type pair, and KM from the per-stage population trackers (one per stage and type). This automates what would be extremely tedious bookkeeping by hand.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using OrdinaryDiffEq
using Plots
```

Multivariate PGFs

In a multi-type network, each type l has its own probability generating function $\psi_l(x_1, \ldots, x_K)$ that describes its connections to all K types. For a type-l node, the variable x_k corresponds to edges connecting to type-k neighbors. The PGF encodes the joint degree distribution — the probability that a randomly chosen type-l node has d_1 type-1 neighbors, d_2 type-2 neighbors, etc.

For Poisson degree distributions (the simplest case), the multivariate PGF takes the form:

$$\psi_l(x_1, \ldots, x_K) = \exp\left(\sum_{k=1}^{K} \kappa_{lk}(x_k - 1)\right)$$

where κ_{lk} is the mean number of type-k neighbors that a type-l node has.

Let's define a two-type network with "Young" and "Old" populations:

```
@parameters κ_YY κ_YO κ_OY κ_OO
pgf_Y = multivariate_poisson_pgf([:Young, :Old], Dict(:Young ⇒ κ_YY, :Old ⇒ κ_YO))
pgf_O = multivariate_poisson_pgf([:Young, :Old], Dict(:Young ⇒ κ_OY, :Old ⇒ κ_OO))
```

```
MultivariatePGF([:Young, :Old], Any[z_Young, z_Old], exp((-1 + z_Old)*κ_OO + (-
1 + z_Young)*κ_OY))
```

Here `κ_YY` is the mean number of Young neighbors a Young individual has, `κ_YO` is the mean number of Old neighbors a Young individual has, and so on.

PGF properties

We can compute mean degrees and partial derivatives symbolically:

```
println("Mean Young neighbors of a Young node: ", mean_degree(pgf_Y, :Young))
println("Mean Old neighbors of a Young node:   ", mean_degree(pgf_Y, :Old))
println("Mean Young neighbors of an Old node:  ", mean_degree(pgf_O, :Young))
println("Mean Old neighbors of an Old node:    ", mean_degree(pgf_O, :Old))
```

```
Mean Young neighbors of a Young node: κ_YY
Mean Old neighbors of a Young node:   κ_YO
Mean Young neighbors of an Old node:  κ_OY
Mean Old neighbors of an Old node:    κ_OO
```

Partial derivatives describe higher-order degree structure:

```
println("∂ψ_Y/∂x_Young = ", partial_derivative(pgf_Y, :Young, 1))
println("∂²ψ_Y/∂x_Young² = ", partial_derivative(pgf_Y, :Young, 2))
println("∂²ψ_Y/(∂x_Young ∂x_Old) = ", mixed_partial(pgf_Y, :Young, :Old))
```

```
∂ψ_Y/∂x_Young = exp((-1 + z_Old)*κ_YO + (-1 + z_Young)*κ_YY)*κ_YY
∂²ψ_Y/∂x_Young² = exp((-1 + z_Old)*κ_YO + (-1 + z_Young)*κ_YY)*(κ_YY^2)
∂²ψ_Y/(∂x_Young ∂x_Old) = exp((-1 + z_Old)*κ_YO + (-1 + z_Young)*κ_YY)*κ_YO*κ_YY
```

We can also evaluate the PGF at specific points:

```
println("ψ_Y(1, 1) = ", eval_multivariate_pgf(pgf_Y, Dict(:Young ⇒ 1, :Old ⇒ 1)))
```

```
ψ_Y(1, 1) = 1
```

Two-type SIR model

Now we combine the multivariate PGFs with a disease progression to build a multi-type edge-based model. We'll use a standard SIR progression:

```
@parameters β γ
progression = DiseaseProgression(
    [
        DiseaseStage(:I; transmission_rate = β),
        DiseaseStage(:R; transmission_rate = 0),
    ],
    [DiseaseTransition(:I, :R, γ)];
    entry = :I,
)
```

```
DiseaseProgression(:S, :I, DiseaseStage[DiseaseStage(:I, β), DiseaseStage(:R, 0)], DiseaseTr
```

Building the model

The `MultiTypeConfigurationModel` constructor takes the list of types, one PGF per type, and the disease progression:

```
model = MultiTypeConfigurationModel(
    types = [:Young, :Old],
    pgfs = Dict(:Young ⇒ pgf_Y, :Old ⇒ pgf_O),
    progression = progression,
)
result = build_edge_system(model)
```

```
EdgeModelSystem(Model multitype_ebm:
Equations (16):
  16 standard: see equations(multitype_ebm)
Unknowns (16): see unknowns(multitype_ebm)
  pop_R_Old(t)
  pop_I_Old(t)
  pop_R_Young(t)
  pop_I_Young(t)
  □
Parameters (8): see parameters(multitype_ebm)
  κ_YY
  κ_YO
  ρ_Young
  κ_OO
  □
Observed (8): see observed(multitype_ebm), Dict{Symbol, Any}(:φ_I_Old_Old => φ_I_Old_Old(t),
1 + θ_Old_Young(t))*κ_YO + (-1 + θ_Young_Young(t))*κ_YY)), (entry = pop_I_Old(t), susceptible
1 + θ_Old_Old(t))*κ_OO + (-1 + θ_Young_Old(t))*κ_OY))], :edge_seed_groups => Any[(entry = φ_I_
1 + θ_Old_Young(t))*κ_YO + (-1 + θ_Young_Young(t))*κ_YY)*(1 - ρ_Young)), (entry = φ_I_Young_Ol
1 + θ_Old_Young(t))*κ_YO + (-1 + θ_Young_Young(t))*κ_YY)*(1 - ρ_Young)), (entry = φ_I_Old_Your
1 + θ_Old_Old(t))*κ_OO + (-1 + θ_Young_Old(t))*κ_OY)*(1 - ρ_Old)), (entry = φ_I_Old_Old(t), ph
1 + θ_Old_Old(t))*κ_OO + (-1 + θ_Young_Old(t))*κ_OY)*(1 - ρ_Old))]))
```

Let's inspect what the model builder generated:

```
n_eqs = length(ModelingToolkit.equations(result.system))
println("Number of equations: ", n_eqs)
```

```
Number of equations: 16
```

With $K = 2$ types and $M = 2$ disease stages (I, R), we expect $K^2(1+M) + KM = 4 \times 3 + 2 \times 2 = 16$ equations.

```
println("State variables:")
for (name, var) in sort(collect(result.variables), by = x -> string(x[1]))
    println("  ", name, " → ", var)
end
```

```
State variables:
  R_Old → pop_R_Old(t)
  R_Young → pop_R_Young(t)
  pop_I_Old → pop_I_Old(t)
  pop_I_Young → pop_I_Young(t)
  pop_R_Old → pop_R_Old(t)
  pop_R_Young → pop_R_Young(t)
  θ_Old_Old → θ_Old_Old(t)
  θ_Old_Young → θ_Old_Young(t)
  θ_Young_Old → θ_Young_Old(t)
  θ_Young_Young → θ_Young_Young(t)
  φ_I_Old_Old → φ_I_Old_Old(t)
  φ_I_Old_Young → φ_I_Old_Young(t)
  φ_I_Young_Old → φ_I_Young_Old(t)
  φ_I_Young_Young → φ_I_Young_Young(t)
  φ_R_Old_Old → φ_R_Old_Old(t)
  φ_R_Old_Young → φ_R_Old_Young(t)
  φ_R_Young_Old → φ_R_Young_Old(t)
  φ_R_Young_Young → φ_R_Young_Young(t)
```

```
println("\nObservable quantities:")
for (name, obs) in sort(collect(result.observables), by = x -> string(x[1]))
    println("  ", name)
end
```

```

Observable quantities:
```

```
I_Old
I_Young
S_Old
S_Young
edge_hazard_Old_Old
edge_hazard_Old_Young
edge_hazard_Young_Old
edge_hazard_Young_Young
excess_hazard_Old_Old
excess_hazard_Old_Young
excess_hazard_Young_Old
excess_hazard_Young_Young
φ_S_Old_Old
φ_S_Old_Young
φ_S_Young_Old
φ_S_Young_Young
```

The variables follow a naming convention:

- θ_{jl} (`θ_Young_Young`, `θ_Old_Young`, etc.): probability that an edge from type l to type j has not transmitted infection
- $\phi_{m,jl}$ (`φ_I_Young_Young`, `φ_R_Old_Young`, etc.): probability that a type-j neighbor reached via a type-l edge is in disease stage m
- R_l (`R_Young`, `R_Old`): fraction of type-l population that has recovered

Solving the two-type model

We assign numeric parameter values representing a contact structure where Young individuals have more contacts (especially with each other), and Old individuals have fewer contacts overall:

Parameter	Value	Meaning
κ_{YY}	6	Young ? Young contacts
κ_{YO}	2	Young ? Old contacts
κ_{OY}	2	Old ? Young contacts
κ_{OO}	4	Old ? Old contacts
γ	0.25	Recovery rate
T	0.75	Edge transmissibility
β	0.75	Transmission rate ($\beta = T\gamma/(1-T)$)

Canonical anchor. This vignette uses $\gamma = 0.25$ and per-edge transmissibility $T = \beta/(\beta+\gamma) = 0.75$, giving $\beta = T\gamma/(1-T) = 0.75$. $R_0 = T\,\rho(M)$ where M is the next-generation matrix derived from the contact structure; preserving T preserves R_0.

```
κ_YY_val = 6.0
κ_YO_val = 2.0
κ_OY_val = 2.0
κ_OO_val = 4.0
γ_val = 0.25
T_val = 0.75
β_val = T_val * γ_val / (1 - T_val)
ε = 1e-2
tspan = (0.0, 40.0)
```

```
(0.0, 40.0)
```

Set up initial conditions and parameters:

```
ic = default_initial_conditions(result; ε = ε)
params = Dict(
    κ_YY ⇒ κ_YY_val, κ_YO ⇒ κ_YO_val,
    κ_OY ⇒ κ_OY_val, κ_OO ⇒ κ_OO_val,
    β ⇒ β_val, γ ⇒ γ_val,
)

prob = ODEProblem(result.system, merge(ic, params), tspan)
sol = solve(prob, Tsit5())
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 36-element Vector{Float64}:
  0.0
  0.060039765733896515
  0.14282733861586347
  0.24274986797517914
  0.36408603488411534
  0.5060709234989873
  0.673489120962293
  0.8786865142897695
  1.1186608510604406
  1.3823346306463933
```

```
 
 17.59921342279818
 19.977440508787414
 22.658507728824542
 25.55605117361315
 28.683405108036986
 32.036573484418106
 35.67004371256627
 39.666961217933
 40.0
u: 36-element Vector{Vector{Float64}}:
 [0.0, 0.01, 0.0, 0.01, 0.0, 0.0, 0.0, 0.0, 0.010000000000000009, 0.010000000000000009, 0.0100
 [0.000170616740808346, 0.012845098375940186, 0.00017817679932437828, 0.013914202290208478,
 [0.000487418181802830444, 0.01805149383853212, 0.0005381318344922485, 0.021310924520375907, 0
 [0.0010438042331261492, 0.027086560409221153, 0.0012240373299151064, 0.034526609396907985,
 [0.002102689877925381, 0.04405112773890065, 0.002624533778474403, 0.059844243356546555, 0.0
 [0.004196161695303761, 0.07669902699762712, 0.00554360095602505 8, 0.10876767644465228, 0.00
 [0.008650402251937245, 0.14174441242062438, 0.011929811113655204, 0.20391628139757084, 0.00
 [0.018991040255172533, 0.27015146649169414, 0.02662605965825947, 0.3774741588597058, 0.0162
 [0.04087378864614523, 0.4607441795006191, 0.0559795624620785, 0.5940321030249053, 0.0339739
 [0.07727110008239296, 0.6298674744023615, 0.10050970223097717, 0.7362109014896945, 0.061724
 
 [0.9720360711767358, 0.016531014065338384, 0.9816043537589656, 0.015860885962118, 0.2471417
7,    1.8357946915521578e-7,    1.0196624573053671e-7,    1.019662457305337e-
7, 0.2585748237530466, 0.2585748237530466, 0.251901146683872, 0.2519011466838719]
 [0.9794450453183347, 0.009122046962477304, 0.9887129837717261, 0.008752257551576702, 0.2471
8,    2.878990513160996e-8,    1.6504711554936675e-8,    1.650471155493419e-
8, 0.25857470238182023, 0.25857470238182023, 0.25190108138605694, 0.2519010813860569]
 [0.9839003710770736, 0.004666722257965245, 0.9929876992341905, 0.004477542330296816, 0.2471
9, 5.670299339180299e-9, 3.577904269818757e-9, 3.577904269818334e-9, 0.2585746842514457, 0.2
 [0.986305378693601, 0.00226171483499365, 0.9952952123924849, 0.0021700292174692626, 0.24714
9,    1.4881372525058022e-9,    1.08054284334654e-9,    1.0805428433464995e-
9, 0.25857468096965724, 0.25857468096965724, 0.2519010696029419, 0.25190106960294184]
 [0.9875321189343048, 0.0010349746377994516, 0.9964722228907477, 0.000993018729704927, 0.247
10,    5.627033088227304e-10,    4.900822681310389e-10,    4.900822681310184e-
10, 0.2585746802429496, 0.2585746802429496, 0.2519010691522226, 0.25190106915222255]
 [0.988119450808836, 0.00044764277499854694, 0.9970357454332898, 0.0004294961899349136, 0.24
10,   3.1012386314504056e-10,   3.369951221056154e-10,   3.3699512210560484e-
10, 0.2585746800447173, 0.2585746800447173, 0.25190106903532816, 0.2519010690353281]
 [0.9883865606029006, 0.00018053298146526495, 0.9972920270965161, 0.00017321452631120408, 0.
10,    2.71070970897244e-10,    3.847617880566309e-10,    3.84761788056609e-
10, 0.2585746800150291, 0.2585746800150291, 0.25190106907145116, 0.2519010690714511]
 [0.9885005886193388, 6.650495217327618e-5, 0.9974014326281286, 6.380898913797419e-
```

```
5, 0.247141773289224, 0.247141773289224, 0.24936631020020492, 0.24936631020020497, 4.146157
10,    4.146157442885559e-10,    8.16446548029776e-10,    8.164465480297159e-
10, 0.2585746801323281, 0.2585746801323281, 0.2519010693993852, 0.25190106939938517]
 [0.9885059015561807, 6.119202337161467e-5, 0.9974065301901152, 5.8711430234317364e-
5, 0.24714177331890028, 0.24714177331890028, 0.24936631025814215, 0.2493663102581422, 3.0395
10,   3.0395083113494153e-10,   5.877804586669322e-10,   5.877804586668892e-
10, 0.25857468004329925, 0.25857468004329925, 0.2519010692255735, 0.2519010692255734]
```

Extracting population trajectories

For Poisson PGFs, we can compute the susceptible fraction of each type from the θ values. For type l:

$$S_l(t) = \psi_l\big(\theta_{1l}(t), \ldots, \theta_{Kl}(t)\big) = \exp\left(\sum_{k=1}^{K} \kappa_{lk}\big(\theta_{kl}(t) - 1\big)\right)$$

```
# Extract θ trajectories
θ_YY = compartment(sol, result, :θ_Young_Young)
θ_OY = compartment(sol, result, :θ_Old_Young)
θ_YO = compartment(sol, result, :θ_Young_Old)
θ_OO = compartment(sol, result, :θ_Old_Old)

# Compute S for each type using the Poisson PGF formula
S_Young = exp.(κ_YY_val .* (θ_YY .- 1) .+ κ_YO_val .* (θ_OY .- 1))
S_Old   = exp.(κ_OY_val .* (θ_YO .- 1) .+ κ_OO_val .* (θ_OO .- 1))

# Extract R for each type
R_Young = compartment(sol, result, :R_Young)
R_Old   = compartment(sol, result, :R_Old)

# I = 1 - S - R
I_Young = 1.0 .- S_Young .- R_Young
I_Old   = 1.0 .- S_Old .- R_Old
```

```
36-element Vector{Float64}:
  0.0
  0.002875560127535618
  0.008137745191994101
  0.017269693266252235
  0.03441630122532625
  0.06741503478371005
```

```
 0.13316197368578953
 0.2629696294017455
 0.45571102831179244
 0.6269002820387681
 □
 0.016423494838608388
 0.009014527806844086
 0.0045592031129805255
 0.002154195691964067
 0.0009274554952092284
 0.0003401236325267787
 7.301383899893654e-5
-4.101419042290555e-5
-4.632711914343002e-5
```

```
plot(sol.t, S_Young, label="S Young", lw=2, color=1)
plot!(sol.t, I_Young, label="I Young", lw=2, color=2)
plot!(sol.t, R_Young, label="R Young", lw=2, color=3)
plot!(sol.t, S_Old, label="S Old", lw=2, ls=:dash, color=1)
plot!(sol.t, I_Old, label="I Old", lw=2, ls=:dash, color=2)
plot!(sol.t, R_Old, label="R Old", lw=2, ls=:dash, color=3)
xlabel!("Time")
ylabel!("Fraction")
title!("Two-type SIR: Young (solid) vs Old (dashed)")
```

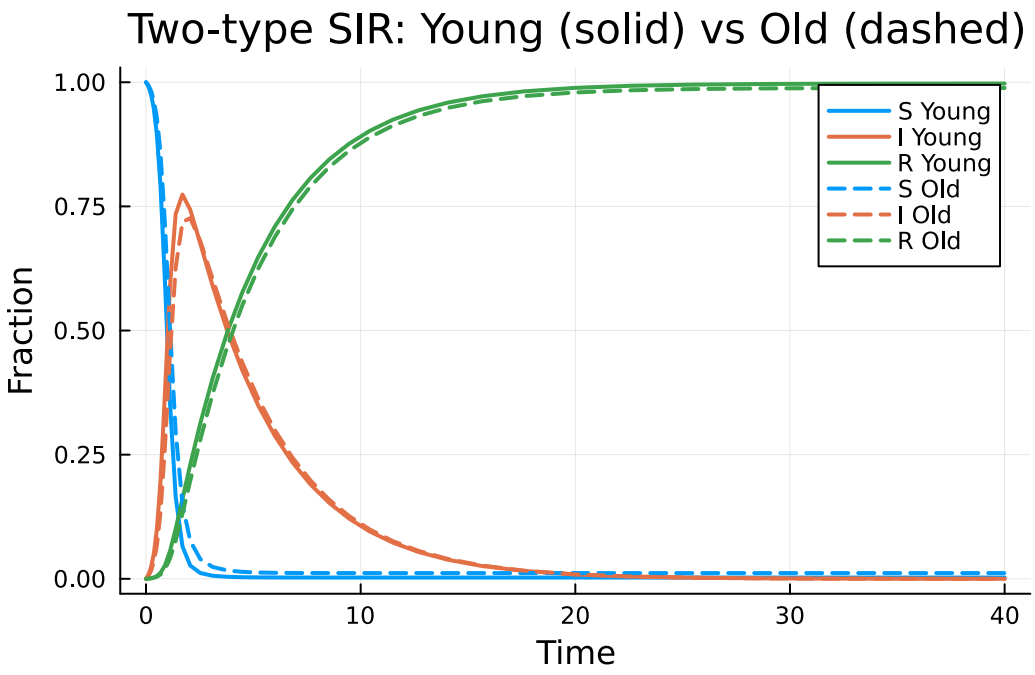

Figure 1: SIR dynamics in a two-type (Young/Old) population. Young individuals have more contacts and experience a faster, larger epidemic.

The Young population (solid lines) experiences a faster epidemic wave because they have more total contacts ($\kappa_{YY} + \kappa_{YO} = 8$) compared to the Old population ($\kappa_{OY} + \kappa_{OO} = 6$, dashed lines).

```
plot(sol.t, I_Young, label="I Young", lw=2, color=1)
plot!(sol.t, I_Old, label="I Old", lw=2, color=2)
xlabel!("Time")
ylabel!("Prevalence")
title!("Infection prevalence by type")
```

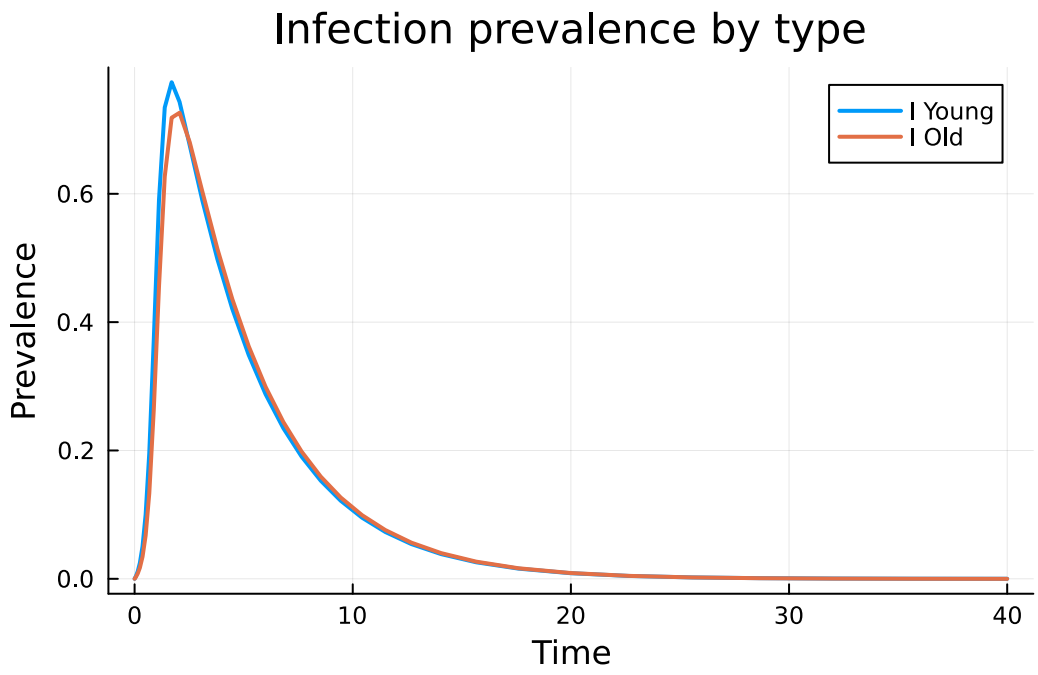

Figure 2: Prevalence (infected fraction) comparison between Young and Old populations.

Assortative mixing with a contact matrix

By default, the multi-type model assumes homogeneous mixing — the transmission rate across an edge depends only on the disease state of the infector. In reality, mixing is often assortative: individuals preferentially contact others of the same type.

The `contact_matrix` parameter scales the transmission rate on each type of edge. A value of 1.0 means the baseline transmission rate applies; values less than 1.0 reduce cross-group transmission:

```
model_assort = MultiTypeConfigurationModel(
    types = [:Young, :Old],
    pgfs = Dict(:Young ⇒ pgf_Y, :Old ⇒ pgf_O),
    progression = progression,
    contact_matrix = Dict(
        (:Young, :Young) ⇒ 1.0,
        (:Old, :Old) ⇒ 1.0,
        (:Young, :Old) ⇒ 0.5,
        (:Old, :Young) ⇒ 0.5,
    ),
```

```
)
result_assort = build_edge_system(model_assort)
```

```
EdgeModelSystem(Model multitype_ebm:
Equations (16):
  16 standard: see equations(multitype_ebm)
Unknowns (16): see unknowns(multitype_ebm)
  pop_R_Old(t)
  pop_I_Old(t)
  pop_R_Young(t)
  pop_I_Young(t)
  ⋮
Parameters (8): see parameters(multitype_ebm)
  κ_YY
  κ_YO
  ρ_Young
  κ_OO
  ⋮
Observed (8): see observed(multitype_ebm), Dict{Symbol, Any}(:φ_I_Old_Old => φ_I_Old_Old(t),
1 + θ_Old_Young(t))*κ_YO + (-1 + θ_Young_Young(t))*κ_YY)), (entry = pop_I_Old(t), susceptible
1 + θ_Old_Old(t))*κ_OO + (-1 + θ_Young_Old(t))*κ_OY))], :edge_seed_groups => Any[(entry = φ_I_
1 + θ_Old_Young(t))*κ_YO + (-1 + θ_Young_Young(t))*κ_YY)*(1 - ρ_Young)), (entry = φ_I_Young_Ol
1 + θ_Old_Young(t))*κ_YO + (-1 + θ_Young_Young(t))*κ_YY)*(1 - ρ_Young)), (entry = φ_I_Old_Your
1 + θ_Old_Old(t))*κ_OO + (-1 + θ_Young_Old(t))*κ_OY)*(1 - ρ_Old)), (entry = φ_I_Old_Old(t), ph
1 + θ_Old_Old(t))*κ_OO + (-1 + θ_Young_Old(t))*κ_OY)*(1 - ρ_Old))]))
```

The contact matrix entry $c_{jl} = 0.5$ for cross-type edges means that a Young infector transmits to an Old neighbor at half the baseline rate (and vice versa). This models reduced cross-group transmission efficiency due to shorter or less intimate contacts.

```
ic_assort = default_initial_conditions(result_assort; ε = ε)
prob_assort = ODEProblem(result_assort.system, merge(ic_assort, params), tspan)
sol_assort = solve(prob_assort, Tsit5())
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 35-element Vector{Float64}:
  0.0
  0.0630919429358846
  0.1551811444688565
  0.2650483578237709
```

```
  0.4008407688826301
  0.5601325273621376
  0.7489955757295332
  0.9795372155732934
  1.2560479793654475
  1.5541195967806887
  
 17.293558431035066
 19.137298981334574
 21.2622804409332
 23.748545480062635
 26.58264043738462
 29.64666803245556
 32.97412240481457
 36.598354566819594
 40.0
u: 35-element Vector{Vector{Float64}}:
 [0.0, 0.01, 0.0, 0.01, 0.0, 0.0, 0.0, 0.0, 0.010000000000000009, 0.010000000000000009, 0.0100
 [0.00017611162505556203, 0.012407883336372851, 0.000184510845865112, 0.013541225922849556,
 [0.0005107899557458589, 0.016879317428765098, 0.0005720843695279234, 0.020539035480131706,
 [0.0010692971439301498, 0.024211200050781104, 0.0012929847631462804, 0.03278414963090179, 0
 [0.0021024060233171176, 0.03760710801929747, 0.002775927743783891, 0.0564615281665531, 0.0
 [0.004056461702881914, 0.06254535808368339, 0.005857211581186529, 0.10212640008988663, 0.00
 [0.008070871303609539, 0.11172454282785142, 0.01261948566644359, 0.19142256374115293, 0.006
 [0.01714729408959921, 0.210867029668486, 0.02812481056842404, 0.3549562329206101, 0.0142948
 [0.03730766231676618, 0.3776272464395356, 0.060265257368735746, 0.5689113617611308, 0.03030
 [0.07215815800862198, 0.5495222777183065, 0.10868321144904904, 0.7106012854472002, 0.056417
 
 [0.9651606818955324, 0.019208179931967915, 0.9787315479670324, 0.017827537430031393, 0.2460
6, 6.47419073940612e-5, 4.677537605221808e-5, 5.969280734310425e-7, 0.2617248240138468, 0.40
 [0.9722545255135842, 0.01211503476849723, 0.9853153947834559, 0.011243883569034255, 0.24609
7,    2.0856351569388908e-5,    1.4904605792079806e-5,    1.707575359135982e-
7, 0.26172326667519574, 0.40939077764169285, 0.4020733757519814, 0.25258066930378476]
 [0.9772474909758396, 0.007122306313599758, 0.9899492915192241, 0.006610053276603022, 0.2460
7,    5.667660019809683e-6,    4.0024122579756234e-6,    4.341770979614041e-
8, 0.2617227646344847, 0.4093815222223483, 0.4020667945698587, 0.2525805239664124]
 [0.980544345215211, 0.003825519854317441, 0.9930090143998356, 0.003550349676589282, 0.24609
8,    1.2481790494737626e-6,    8.705478993593792e-7,    9.388243764086626e-
9, 0.2617226253391985, 0.4093788298657126, 0.40206490388288485, 0.2525804839838645]
 [0.9824862140385832, 0.0018836664197423307, 0.9948112007259444, 0.001748167790085232, 0.246
9,   2.2942748940776265e-7,   1.5824093194521427e-7,   1.7891933595110384e-
9, 0.26172259410608806, 0.4093782093814984, 0.40206447383494154, 0.2525804749548731]
 [0.9834941606175711, 0.0008757226945516662, 0.995746641497499, 0.0008127278515293335, 0.246
```

```
9,    3.831662140917099e-8,    2.6206253902880067e-8,    3.446704456599356e-
10, 0.2617225882988747, 0.40937809300269923, 0.40206439411433553, 0.25258047324673194]
 [0.9839886803253611, 0.0003812034660831985, 0.996205587944916, 0.00035378154523539205, 0.24
10, 5.96288874290566e-9, 4.067947060166406e-9, 9.258850515909816e-11, 0.26172258731242276, 0
 [0.984215790985113, 0.00015409288056566096, 0.9964163613444531, 0.00014300816754160702, 0.2
11,   9.178980210823231e-10,   6.328432534052335e-10,   6.350610019835083e-
11, 0.26172258716158137, 0.40937807023133166, 0.4020643786725094, 0.252580472913634]
 [0.9843040379567248, 6.584592161778044e-5, 0.9964982602418264, 6.11092737876293e-
5, 0.24609247095965614, 0.3937479534962059, 0.3986237477677981, 0.24913984236793088, 3.97179
11,   1.3782765486211145e-10,   9.611862474386626e-11,   4.389031924004457e-
11, 0.2617225871210315, 0.4093780697556911, 0.40206437834830305, 0.2525804728962077]
```

```
# Extract trajectories for the assortative model
θ_YY_a = compartment(sol_assort, result_assort, :θ_Young_Young)
θ_OY_a = compartment(sol_assort, result_assort, :θ_Old_Young)
θ_YO_a = compartment(sol_assort, result_assort, :θ_Young_Old)
θ_OO_a = compartment(sol_assort, result_assort, :θ_Old_Old)

S_Young_a = exp.(κ_YY_val .* (θ_YY_a .- 1) .+ κ_YO_val .* (θ_OY_a .- 1))
S_Old_a   = exp.(κ_OY_val .* (θ_YO_a .- 1) .+ κ_OO_val .* (θ_OO_a .- 1))
R_Young_a = compartment(sol_assort, result_assort, :R_Young)
R_Old_a   = compartment(sol_assort, result_assort, :R_Old)
I_Young_a = 1.0 .- S_Young_a .- R_Young_a
I_Old_a   = 1.0 .- S_Old_a .- R_Old_a
```

```
35-element Vector{Float64}:
  0.0
  0.002433984290005555
  0.006953964849552543
  0.01436554760616001
  0.02790719660020925
  0.05311704446697314
  0.10283329119624426
  0.20306804944113782
  0.3717187849980246
  0.5456957597410435
  ?
  0.019057795456768
  0.011964657348942365
  0.006971931288184852
  0.0036751455135708433
  0.0017332922344397428
```

```
  0.00072534853807526 56
  0.00023082931444851074
  3.718729680746158e-6
 -8.452822913918023e-5
```

```
plot(sol.t, I_Young, label="I Young (homogeneous)", lw=2, color=1)
plot!(sol_assort.t, I_Young_a, label="I Young (assortative)", lw=2, ls=:dash, color=1)
plot!(sol.t, I_Old, label="I Old (homogeneous)", lw=2, color=2)
plot!(sol_assort.t, I_Old_a, label="I Old (assortative)", lw=2, ls=:dash, color=2)
xlabel!("Time")
ylabel!("Prevalence")
title!("Homogeneous (solid) vs assortative (dashed) mixing")
```

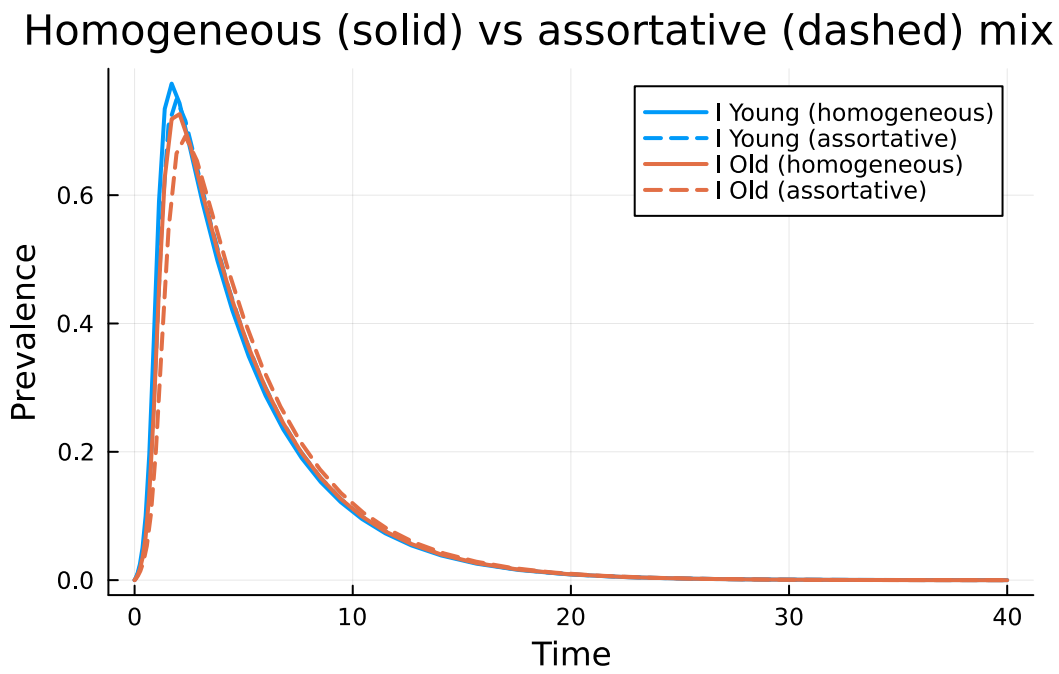

Figure 3: Effect of assortative mixing on epidemic dynamics. Assortative mixing (dashed) slows the epidemic and reduces final size compared to homogeneous mixing (solid).

Assortative mixing tends to slow the epidemic because cross-group transmission is a major driver of epidemic spread — reducing it isolates the two populations, making the overall dynamics more like two separate, smaller epidemics.

```
final_R_Young_hom = R_Young[end]
final_R_Old_hom = R_Old[end]
final_R_Young_assort = R_Young_a[end]
final_R_Old_assort = R_Old_a[end]

bar_labels = ["Young\n(homogeneous)", "Young\n(assortative)", "Old\n(homogeneous)", "Old\n
bar_vals = [final_R_Young_hom, final_R_Young_assort, final_R_Old_hom, final_R_Old_assort]
bar(bar_labels, bar_vals, legend=false, ylabel="Final size (R∞)",
    title="Final epidemic size by type and mixing", color=[1 1 2 2],
    ylim=(0, 1))
```

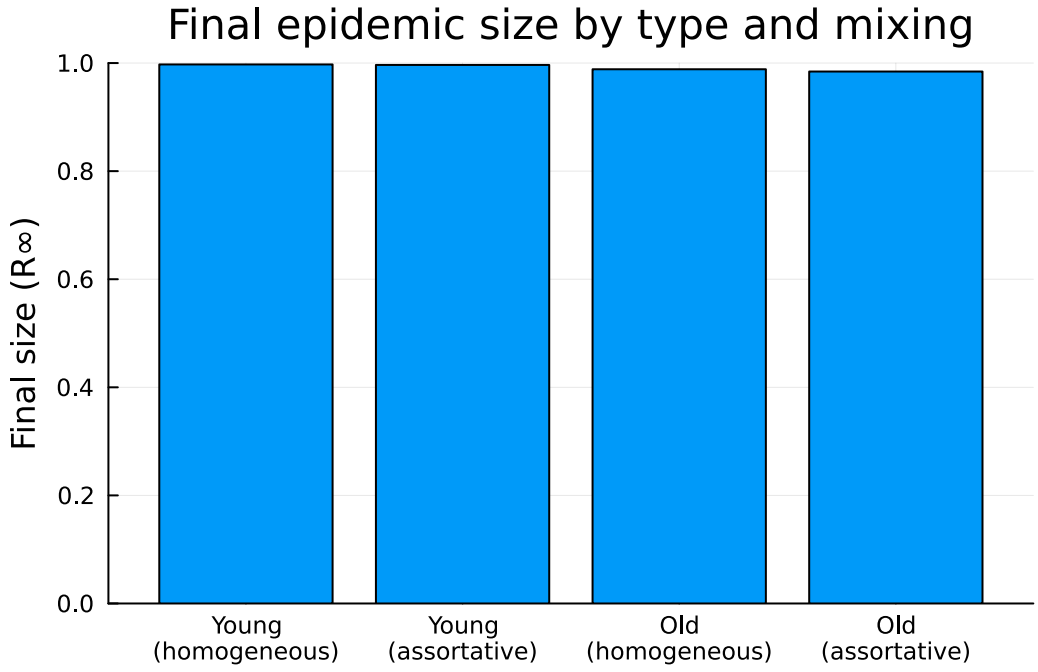

Figure 4: Final epidemic size (recovered fraction) under homogeneous vs assortative mixing.

Scaling to more types

The multi-type framework scales to any number of population types. For example, consider a three-type (Young/Middle/Old) SEIR model:

```
@parameters σ

progression_seir = DiseaseProgression(
```

```
    [
        DiseaseStage(:E; transmission_rate = 0),
        DiseaseStage(:I; transmission_rate = β),
        DiseaseStage(:R; transmission_rate = 0),
    ],
    [
        DiseaseTransition(:E, :I, σ),
        DiseaseTransition(:I, :R, γ),
    ];
    entry = :E,
)
```

```
DiseaseProgression(:S, :E, DiseaseStage[DiseaseStage(:E, 0), DiseaseStage(:I, β), DiseaseSta
```

```
@parameters κ_YY3 κ_YM κ_YO3 κ_MY κ_MM κ_MO κ_OY3 κ_OM κ_OO3
types_3 = [:Y, :M, :O]

pgf_Y3 = multivariate_poisson_pgf(types_3, Dict(:Y ⇒ κ_YY3, :M ⇒ κ_YM, :O ⇒ κ_YO3))
pgf_M3 = multivariate_poisson_pgf(types_3, Dict(:Y ⇒ κ_MY, :M ⇒ κ_MM, :O ⇒ κ_MO))
pgf_O3 = multivariate_poisson_pgf(types_3, Dict(:Y ⇒ κ_OY3, :M ⇒ κ_OM, :O ⇒ κ_OO3))

model_3type = MultiTypeConfigurationModel(
    types = types_3,
    pgfs = Dict(:Y ⇒ pgf_Y3, :M ⇒ pgf_M3, :O ⇒ pgf_O3),
    progression = progression_seir,
)
result_3type = build_edge_system(model_3type)

n_eqs_3 = length(ModelingToolkit.equations(result_3type.system))
println("Three-type SEIR: $n_eqs_3 equations automatically generated")
```

```
Three-type SEIR: 45 equations automatically generated
```

With $K = 3$ types and $M = 3$ stages (E, I, R), we expect $K^2(1 + M) + KM = 9 \times 4 + 3 \times 3 = 45$ equations. Writing these by hand would be impractical and error-prone — the package handles all the symbolic bookkeeping automatically.

```
println("\nState variables (", length(result_3type.variables), " total):")
for (name, _) in sort(collect(result_3type.variables), by = x → string(x[1]))
    println("  ", name)
end
```

```
State variables (48 total):
  R_M
  R_O
  R_Y
  pop_E_M
  pop_E_O
  pop_E_Y
  pop_I_M
  pop_I_O
  pop_I_Y
  pop_R_M
  pop_R_O
  pop_R_Y
  θ_M_M
  θ_M_O
  θ_M_Y
  θ_O_M
  θ_O_O
  θ_O_Y
  θ_Y_M
  θ_Y_O
  θ_Y_Y
  φ_E_M_M
  φ_E_M_O
  φ_E_M_Y
  φ_E_O_M
  φ_E_O_O
  φ_E_O_Y
  φ_E_Y_M
  φ_E_Y_O
  φ_E_Y_Y
  φ_I_M_M
  φ_I_M_O
  φ_I_M_Y
  φ_I_O_M
  φ_I_O_O
  φ_I_O_Y
  φ_I_Y_M
  φ_I_Y_O
  φ_I_Y_Y
  φ_R_M_M
  φ_R_M_O
```

```
φ_R_M_Y
φ_R_O_M
φ_R_O_O
φ_R_O_Y
φ_R_Y_M
φ_R_Y_O
φ_R_Y_Y
```

Summary

Multi-type edge-based models capture the essential features of heterogeneous populations:

- Structured contact patterns: Different types can have different degree distributions and mixing preferences.
- Type-specific dynamics: Each population type experiences its own epidemic trajectory, determined by both its own contact structure and cross-group transmission.
- Contact matrices: Assortative or disassortative mixing can be specified to modulate cross-group transmission rates.
- Automatic scaling: The package generates $K^2(1+M)+KM$ equations for any number of types K and disease stages M, handling all the symbolic algebra and bookkeeping automatically.

Simulation validation

We validate the deterministic two-type EBCM against direct stochastic simulation on a stochastic block model whose within- and between-type mean degrees match $\kappa_{YY}, \kappa_{YO}, \kappa_{OY}, \kappa_{OO}$. Each node is assigned a type (1000 Young, 1000 Old). Within-type edges are sampled at probability $\kappa_{ll}/(N_l-1)$ and between-type edges at κ_{lk}/N_k, giving the prescribed mixing matrix in expectation. We then run an ensemble of `NetworkOutbreaks.jl` `DirectSSA` epidemics with the type-aware compartments $\{S_l, I_l, R_l\}$ for $l \in \{Y, O\}$.

```
include("../_validation.jl")

sbm = sbm_typed_graph_builder(
    [:Young ⇒ 1000, :Old ⇒ 1000],
    Dict((:Young, :Young) ⇒ κ_YY_val, (:Young, :Old) ⇒ κ_YO_val,
         (:Old,   :Young) ⇒ κ_OY_val, (:Old,   :Old) ⇒ κ_OO_val))

tgrid_v, mean_v, std_v = gillespie_multitype_ribbon(
    [:Young, :Old], sbm;
    β = β_val, γ = γ_val, n_graphs = 3, nsims_per_graph = 12,
```

```
    tspan = tspan, seed_fraction = ε,
    tgrid = collect(0.0:1.0:40.0))
```

```
([0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0 … 31.0, 32.0, 33.0, 34.0, 35.0, 36.0, 37.0,
5, 5.555555555555556e-5, 2.777777777777778e-5, 0.0, 0.0, 0.0], (:S, :Old) => [0.99, 0.6479444
5, 5.555555555555556e-5, 2.777777777777778e-5, 2.777777777777778e-5, 2.777777777777778e-
5], (:R, :Old) => [0.0, 0.02688888888888889, 0.17838888888888887, 0.34650000000000003, 0.4888
18, 0.050430999536849506, 0.015061777546396806, 0.01666723808544253, 0.014252039675802028, 0
18, 0.059583368298358044, 0.014468438557277514, 0.016246586943157868, 0.016808798716318612,
```

```
plot(sol.t, I_Young, label="I Young (EBM)", lw=2, color=:red)
plot!(tgrid_v, mean_v[(:I, :Young)],
      ribbon = std_v[(:I, :Young)],
      label = "I Young (SSA mean ± 1σ)",
      color = :red, fillalpha = 0.2, linealpha = 0.6, lw = 1)
plot!(sol.t, I_Old, label="I Old (EBM)", lw=2, color=:blue)
plot!(tgrid_v, mean_v[(:I, :Old)],
      ribbon = std_v[(:I, :Old)],
      label = "I Old (SSA mean ± 1σ)",
      color = :blue, fillalpha = 0.2, linealpha = 0.6, lw = 1)
xlabel!("Time")
ylabel!("Prevalence")
title!("Multi-type SIR: EBM vs SSA on SBM")
```

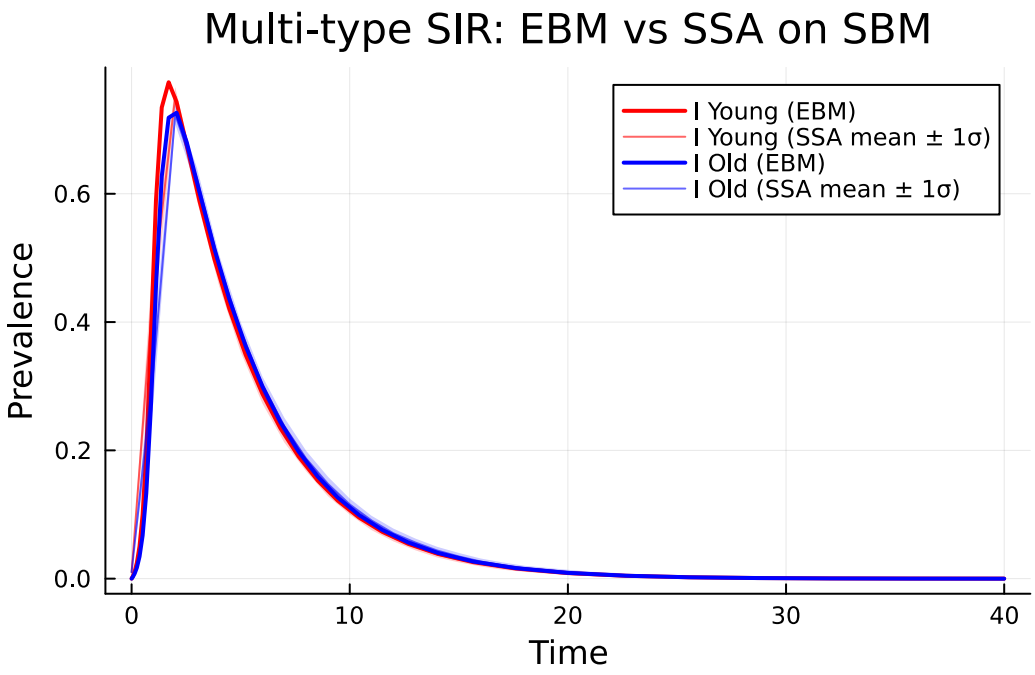

Figure 5: EBM (lines) versus DirectSSA on a stochastic block model with prescribed mixing matrix (ribbons = mean ± 1σ across 36 simulations on 3 graphs, N=2000).

The deterministic curves track the SSA ensemble means closely across both populations, with the Young population peaking earlier and higher because of its larger total mean degree ($\kappa_{YY}+\kappa_{YO} = 8$ vs. $\kappa_{OY} + \kappa_{OO} = 6$).